

Masadan Sipari■

QR Table-Ordering Platform — Project Report

Next.js 16 · Prisma 6 · SQLite (Postgres-ready) · SSE realtime
Prepared 2026-07-29

1. What This Is
2. Architecture & Stack
3. Data Model
4. Core Flows & Surfaces
5. Security & Anti-Abuse Design
6. Production Readiness — What's Critical to Make It Work

1. What This Is

A multi-tenant QR table-ordering app. A customer scans a QR code at their table, browses the venue's menu on their own phone (no app install, no account), places an order, and watches its status live. Orders land instantly on a kitchen/desk dashboard and are bridged to the venue's existing AKINSOFT point-of-sale setup via ESC/POS kitchen printing.

2. Architecture & Stack

- **Frontend / backend** — Next.js App Router (TypeScript), Tailwind. One deployable unit, no separate API server.
- **Database** — Prisma 6, SQLite for local dev (`prisma/dev.db`); schema is written to be Postgres-compatible (swap `provider` + `DATABASE_URL`).
- **Realtime** — Server-Sent Events (`/api/desk/stream`, `/api/session/stream`) fed by an in-process `EventEmitter` (`src/lib/bus.ts`).
- **Auth** — two separate identity systems: staff (JWT via `jose`, `httpOnly` cookie) and customer table sessions (opaque token, cookie-bound).
- **POS bridge** — `bridge/agent.mjs`, a standalone on-prem Node script that polls the app and prints raw ESC/POS to a LAN printer. Outbound HTTP only — no inbound ports opened at the venue.

3. Data Model (Prisma)

Fully venue-scoped (multi-tenant): `Venue` → `Table` / `Category` / `MenuItem` / `StaffUser` / `TableSession` / `Order`. Notable design choices:

- Money stored as **kurumi integers**, never floats.
- `OrderItem` snapshots `name/price/modifiers` so later menu edits never rewrite order history.
- Order already has `paymentStatus` / `Provider` / `Ref` columns — unused today (pay-at-till) but ready for a future PSP integration.
- `OrderAttempt` is a full abuse ledger — logs every rejected attempt (bad QR signature, expired session, rate limit) with IP/user-agent.
- `PosDelivery` outbox decouples order acceptance from POS printing — the customer→desk flow never blocks on a printer being reachable.

4. Core Flows & Surfaces

URL	Who	What
-----	-----	------

/scan/{code}?k={sig}	Customer (via QR)	Verify signature → mint table session
/t/{code}	Customer	Menu, cart, modifiers, live order status (TR/EN)
/desk	Kitchen staff	Live ticket board via SSE, accept/reject/status, printer-failure badges
/admin	Owner	Menu CRUD, photo upload, 86 toggle, table+QR mgmt, session kill, CSV ex

On order acceptance: ticket → PosDelivery outbox → rendered by Venue.posAdapter (console for dev, escpos_bridge as the primary path — the bridge agent claims and prints it).

5. Security & Anti-Abuse Design

These are deliberate, previously-confirmed decisions — not open questions.

- **Signed QR** — $k = \text{HMAC}(\text{qrSecret}, \text{tableCode}:\text{qrVersion})$, verified with `timingSafeEqual`. Regenerating a QR bumps `qrVersion` and instantly kills old photos/screenshots of the code.
- **Table-bound sessions** — 2h hard cap, 30min idle timeout, max 6 concurrent per table, staff-revocable.
- **Economic backstop** — pay-at-till, every order needs staff accept, first order of a session is badged “YEN■ OTURUM”.
- **Rate limits + attempt ledger** for ongoing abuse analysis.
- Geo/IP checks are **intentionally deferred**, not missing — an advisory layer planned for later, not a gap.

6. Production Readiness — What's Critical to Make It Work

The architecture is sound end-to-end, but several pieces are explicitly dev-mode placeholders. These are the load-bearing gaps to close before this runs for a real venue.

■ AUTH_SECRET has a hardcoded dev fallback

`src/lib/staffAuth.ts:6` falls back to the literal string "dev-secret" if the env var is unset. If this ships unset, every staff JWT (admin/desk login) is forgeable. This is the single most important check before go-live.

■ Realtime bus is single-process only

`src/lib/bus.ts` uses a plain in-process EventEmitter to feed the desk board and session-status SSE streams. On any multi-instance or serverless host (including Vercel by default), live updates will silently break across instances. Needs Redis pub/sub (e.g. Upstash Redis via the Vercel Marketplace) behind the same publish/subscribe signature before deploying.

■ Photo uploads write to local disk

`src/app/api/admin/upload/route.ts` writes menu photos to `public/uploads`. Serverless filesystems (Vercel included) are ephemeral/read-only per invocation — uploaded photos would not persist. Needs Vercel Blob or S3 before the admin photo upload is usable in hosted production.

■ SQLite is dev-only

The schema is written to be Postgres-compatible, but `DATABASE_URL/provider` must be swapped (e.g. Neon Postgres via the Vercel Marketplace) before any real deployment — a SQLite file does not survive serverless deploys.

■ NEXT_PUBLIC_BASE_URL must match the printed-QR domain exactly

Every physical table QR is generated against this URL at print time. Get it wrong once and every printed QR in the venue is dead until reprinted.

Additional gaps, lower urgency:

- No automated tests exist in the repo — session/QR-signature/rate-limit logic is security-critical and currently relies on manual verification.

- AKINSOFT/Wolvox integration is still unconfirmed — the bridge today prints generic ESC/POS to any network printer on port 9100, not verified against the venue's actual AKINSOFT module or license.

Already-decided, not gaps:

- Pay-at-till only — PSP integration (iyzico/PayTR) intentionally deferred; schema is ready for it.
- Geo/IP anti-abuse checks intentionally deferred to a later advisory layer.